trading

November 4, 2026

1 Trading

The purpose of this worksheet is to demonstrate and describe how trading works in OSEM.

1.1 Table of contents

1. What trading does
2. Setting up a run
3. The projection loop
4. Trading period by period
   - What was traded and when
   - How much was traded
5. Checks
   - Cash is swept to zero
   - Trading does not create or destroy value

1.2 1. What trading does

OSEM holds cash in a single bank account. Over a period that account collects the dividends, coupons and matured notionals the portfolio paid out, and is drained by the liability outflows the company committed to. At the end of the period the balance is almost never zero, and `trade` in `MainLoop.py` puts it back to zero by resizing the portfolio.

The rule is **proportional**: every position is scaled by the same factor, so the asset allocation is preserved and only the size of the portfolio changes. Writing C for the bank balance and MV for the market value of the portfolio at that moment:

- if $C > 0$ the surplus buys more of everything, each position scaled by $1 + C/MV$
- if $C < 0$ the deficit is covered by selling, each position scaled by $1 - \min(1, -C/MV)$
- if $MV \leq 0$ there is nothing to trade and the balance is left alone

The bank account is then credited with the market value that left the portfolio, or debited with the value that entered it:

$$C^{after} = C^{before} + MV^{before} - MV^{after}$$

Both branches therefore drive the balance to zero, and the total of cash plus portfolio is unchanged by the trade itself. The one exception is a deficit larger than the whole portfolio: the sell fraction is capped at 1, everything is sold, and the account stays negative.

Note what trading does **not** do: it never changes the mix between equities and bonds, and it never picks individual assets. Rebalancing towards a target allocation is not modelled.

1.3 2. Setting up a run

The same inputs `main.py` builds before entering its loop. They come from `notebooks/data/notebooks.ini`, a copy of `ALM.ini` that names a small, fixed input set in `notebooks/data/Input`, so the results do not change with the model's live `Input` folder.

[1]:
```
import os
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
import sys

base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
from osem.ImportData import get_configuration, get_settings, import_SWEiopa,␣
 ↪get_cash, get_equity_share,get_corporate_bonds, get_liability
from osem.ConfigurationClass import Configuration
from osem.CurvesClass import Curves
from osem.BondClasses import BondPortfolio
from osem.EquityClasses import EquitySharePortfolio
from osem.MainLoop import (create_cashflow_dataframe, create_liabilities_df,␣
 ↪set_dates_of_interest,
                           process_expired_cf, process_expired_liab, trade,
                           find_terminated_positions, liquidate_positions,
                           portfolio_market_value)
```

[2]:
```
base_folder = os.getcwd()  # Get current working directory
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "notebooks", "data",␣
 ↪"notebooks.ini"), os)
```

[3]:
```
parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
equity_portfolio_file = conf.input_equity_portfolio
liability_cashflow_file = conf.input_liability_cashflow
corporate_bond_portfolio_file = conf.input_corporate_bond_portfolio
```

[4]:
```
settings = get_settings(parameters_file)
```

[5]:
```
[maturities_country, curve_country, extra_param, Qb] =  import_SWEiopa(settings.
 ↪EIOPA_param_file,
                                                                          ␣
 ↪settings.EIOPA_curves_file, settings.country)
```

Curves object with information about term structure:

[6]:
```
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,␣
↪settings.modelling_date,
                    settings.country)
```

[7]:
```
curves.set_observed_term_structure(maturity_vec=curve_country.index.
↪to_numpy(dtype=float), yield_vec=curve_country.values)
curves.calc_fwd_rates()
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

Cash, equities and bonds:

[8]:
```
cash = get_cash(cash_portfolio_file)
```

[9]:
```
equity_input_generator = get_equity_share(equity_portfolio_file) # Create␣
↪generator that contains all equity positions
eq_input = {equity_share.asset_id: equity_share for equity_share in␣
↪equity_input_generator}
```

[10]:
```
bond_input_generator = get_corporate_bonds(corporate_bond_portfolio_file)
bond_input = {corp_bond.asset_id: corp_bond for corp_bond in␣
↪bond_input_generator}
```

[11]:
```
eq_ptf = EquitySharePortfolio(eq_input)
bd_ptf = BondPortfolio(bond_input)
```

[12]:
```
div_dict = eq_ptf.create_dividend_flows(settings.modelling_date, settings.
↪end_date)
ter_dict = eq_ptf.create_terminal_flows(modelling_date=settings.modelling_date,
                                                                    ␣
↪terminal_date=settings.end_date)
cpn_flows = bd_ptf.create_coupon_flows(settings.modelling_date, settings.
↪end_date)
not_flows = bd_ptf.create_maturity_flows(terminal_date=settings.end_date)
```

[13]:
```
unique_div_dates = eq_ptf.unique_dates_profile(div_dict)
unique_ter_dates = eq_ptf.unique_dates_profile(ter_dict)
unique_cpn_dates = bd_ptf.unique_dates_profile(cpn_flows)
unique_not_dates = bd_ptf.unique_dates_profile(not_flows)
```

[14]:
```
liabilities = get_liability(liability_cashflow_file)
unique_liabilities_dates = liabilities.unique_dates_profile()
```

[15]:
```
[eq_price_df, eq_growth_df, eq_units_df] = eq_ptf.
↪init_equity_portfolio_to_dataframe(settings.modelling_date)
```

```
[bd_price_df, bd_zspread_df, bd_units_df] = bd_ptf.
 ↪init_bond_portfolio_to_dataframe(settings.modelling_date)
bank_account = pd.DataFrame(data=[cash.bank_account], columns=[settings.
 ↪modelling_date])
```

[16]:
```
div_df = create_cashflow_dataframe(div_dict, unique_div_dates)
ter_df = create_cashflow_dataframe(ter_dict, unique_ter_dates)
cpn_df = create_cashflow_dataframe(cpn_flows, unique_cpn_dates)
not_df = create_cashflow_dataframe(not_flows, unique_not_dates)
liab_df = create_liabilities_df(liabilities)
```

[17]:
```
dates_of_interest = set_dates_of_interest(settings.modelling_date, settings.
 ↪end_date)
previous_date = settings.modelling_date
proj_period = 0
```

[18]:
```
bd_zspread_df = bd_ptf.calibrate_bond_portfolio(bd_zspread_df, settings,␣
 ↪proj_period, curves)
```

1.4 3. The projection loop

The same sequence `main.py` runs, with the state around each `trade` call recorded so the trading itself can be looked at afterwards.

Two details matter for the numbers to come out right:

- positions whose terminal value has just been paid out are **liquidated** before trading. Without that step the cash is credited to the bank account while the position is still in the portfolio, `trade` reinvests it, and the same money is counted twice. Leaving it out inflates the closing portfolio by roughly half.
- the bond repricing is done at projection year `proj_period + 1`, because `proj_period` counts *completed* periods while `current_date` is one year further on.

[19]:
```
trading_log = []

for current_date in dates_of_interest.values:
    eq_units_df[current_date] = eq_units_df[previous_date]
    bd_units_df[current_date] = bd_units_df[previous_date]
    bank_account[current_date] = bank_account[previous_date]

    time_frac = (current_date - previous_date).days / 365.25

    # ---- cash flows that expired during the period ----
    expired_cash, div_df, unique_div_dates =␣
 ↪process_expired_cf(unique_div_dates, current_date, div_df, eq_units_df)
    bank_account[current_date] += expired_cash
```

```
    expired_cash, cpn_df, unique_cpn_dates =␣
↪process_expired_cf(unique_cpn_dates, current_date, cpn_df, bd_units_df)
    bank_account[current_date] += expired_cash

    terminated_eq_ids = find_terminated_positions(unique_ter_dates,␣
↪current_date, ter_df)
    expired_cash, ter_df, unique_ter_dates =␣
↪process_expired_cf(unique_ter_dates, current_date, ter_df, eq_units_df)
    bank_account[current_date] += expired_cash

    matured_bd_ids = find_terminated_positions(unique_not_dates, current_date,␣
↪not_df)
    expired_cash, not_df, unique_not_dates =␣
↪process_expired_cf(unique_not_dates, current_date, not_df, bd_units_df)
    bank_account[current_date] += expired_cash

    expired_cash, liab_df, unique_liabilities_dates =␣
↪process_expired_liab(unique_liabilities_dates, current_date, liab_df)
    bank_account[current_date] -= expired_cash

    # ---- asset growth and repricing ----
    eq_price_df[current_date] = eq_price_df[previous_date] * (
            1 + eq_growth_df[settings.modelling_date]) ** time_frac

    bd_price_df[current_date] = bd_price_df[previous_date]
    bd_price_df = bd_ptf.price_bond_portfolio(cpn_df, not_df, settings,␣
↪proj_period + 1, curves,
                                              bd_zspread_df, bd_price_df,␣
↪current_date)

    # ---- close out positions whose terminal value has been paid ----
    eq_units_df = liquidate_positions(eq_units_df, terminated_eq_ids,␣
↪current_date)
    bd_units_df = liquidate_positions(bd_units_df, matured_bd_ids, current_date)

    # ---- trade, recording the state either side of it ----
    cash_before = float(bank_account[current_date].iloc[0])
    mv_before = portfolio_market_value(eq_price_df, eq_units_df, bd_price_df,␣
↪bd_units_df, current_date)
    eq_units_before = float(eq_units_df[current_date].sum())
    bd_units_before = float(bd_units_df[current_date].sum())

    eq_units_df, bd_units_df, bank_account = trade(
        current_date, bank_account, eq_units_df, eq_price_df, bd_units_df,␣
↪bd_price_df)
```

```
    cash_after = float(bank_account[current_date].iloc[0])
    mv_after = portfolio_market_value(eq_price_df, eq_units_df, bd_price_df,␣
 ↪bd_units_df, current_date)

    trading_log.append({
        "Date": current_date,
        "Cash before": cash_before,
        "Market value before": mv_before,
        # No portfolio left to trade against is its own case, not a buy
        "Action": ("None" if mv_before <= 0 else
                   "Buy" if cash_before > 0 else
                   "Sell" if cash_before < 0 else "None"),
        "Traded fraction": (mv_after / mv_before - 1) if mv_before > 0 else 0.0,
        "Equity units before": eq_units_before,
        "Equity units after": float(eq_units_df[current_date].sum()),
        "Bond units before": bd_units_before,
        "Bond units after": float(bd_units_df[current_date].sum()),
        "Cash after": cash_after,
        "Market value after": mv_after,
    })

    previous_date = current_date
    proj_period += 1

trading_log = pd.DataFrame(trading_log).set_index("Date")
```

1.5 4. Trading period by period

1.5.1 What was traded and when

Each row is one period: the balance the period ended with before trading, the direction that implies, and what the portfolio looked like either side of the trade.

---

[20]:
```
display(trading_log[["Cash before", "Market value before", "Action",
                     "Traded fraction", "Cash after", "Market value after"]].
 ↪head(12))

print(trading_log["Action"].value_counts().to_string())
```

```
              Cash before  Market value before Action  Traded fraction  \
Date
2024-04-29  103122.078061         87239.695662    Buy         1.182055
2025-04-29    7276.802025        191955.888571    Buy         0.037909
2026-04-29    6948.844834        200839.951924    Buy         0.034599
2027-04-29    7512.612660        209473.762935    Buy         0.035864
2028-04-29   31902.182691        194411.729982    Buy         0.164096
2029-04-29    8470.936027        228025.501098    Buy         0.037149
```

```
2030-04-29   67644.537980        178840.985094    Buy          0.378238
2031-04-29   10862.981299        247971.768044    Buy          0.043807
2032-04-29   11427.643572        260459.397793    Buy          0.043875
2033-04-29   56064.595641        229154.055713    Buy          0.244659
2034-04-29   10999.883439        289451.508303    Buy          0.038003
2035-04-29   11504.521851        304974.354339    Buy          0.037723
```

```
              Cash after  Market value after
Date
2024-04-29 -1.455192e-11       190361.773724
2025-04-29  0.000000e+00       199232.690596
2026-04-29  1.637090e-11       207788.796758
2027-04-29  1.273293e-11       216986.375595
2028-04-29 -7.275958e-12       226313.912673
2029-04-29  1.818989e-11       236496.437126
2030-04-29  2.910383e-11       246485.523073
2031-04-29  2.728484e-11       258834.749344
2032-04-29  9.094947e-12       271887.041365
2033-04-29 -1.455192e-11       285218.651353
2034-04-29  3.819878e-11       300451.391741
2035-04-29 -4.729372e-11       316478.876190
```

```
Action
Buy     49
None     1
```

The unit columns show the proportional rule at work. Every position is scaled by the same factor, so the equity and bond unit totals move by the same percentage:

---

[21]:
```
units_view = trading_log[["Equity units before", "Equity units after",
                          "Bond units before", "Bond units after", "Traded␣
 ↪fraction"]].copy()
units_view["Equity change"] = (units_view["Equity units after"]
                               / units_view["Equity units before"] - 1)
units_view["Bond change"] = (units_view["Bond units after"]
                             / units_view["Bond units before"] - 1)

display(units_view[["Equity change", "Bond change", "Traded fraction"]].
 ↪head(12))
```

```
            Equity change  Bond change  Traded fraction
Date
2024-04-29       1.182055     1.182055         1.182055
2025-04-29       0.037909     0.037909         0.037909
2026-04-29       0.034599     0.034599         0.034599
2027-04-29       0.035864     0.035864         0.035864
```

```
2028-04-29       0.164096      0.164096          0.164096
2029-04-29       0.037149      0.037149          0.037149
2030-04-29       0.378238      0.378238          0.378238
2031-04-29       0.043807      0.043807          0.043807
2032-04-29       0.043875      0.043875          0.043875
2033-04-29       0.244659      0.244659          0.244659
2034-04-29       0.038003      0.038003          0.038003
2035-04-29       0.037723      0.037723          0.037723
```

1.5.2 How much was traded

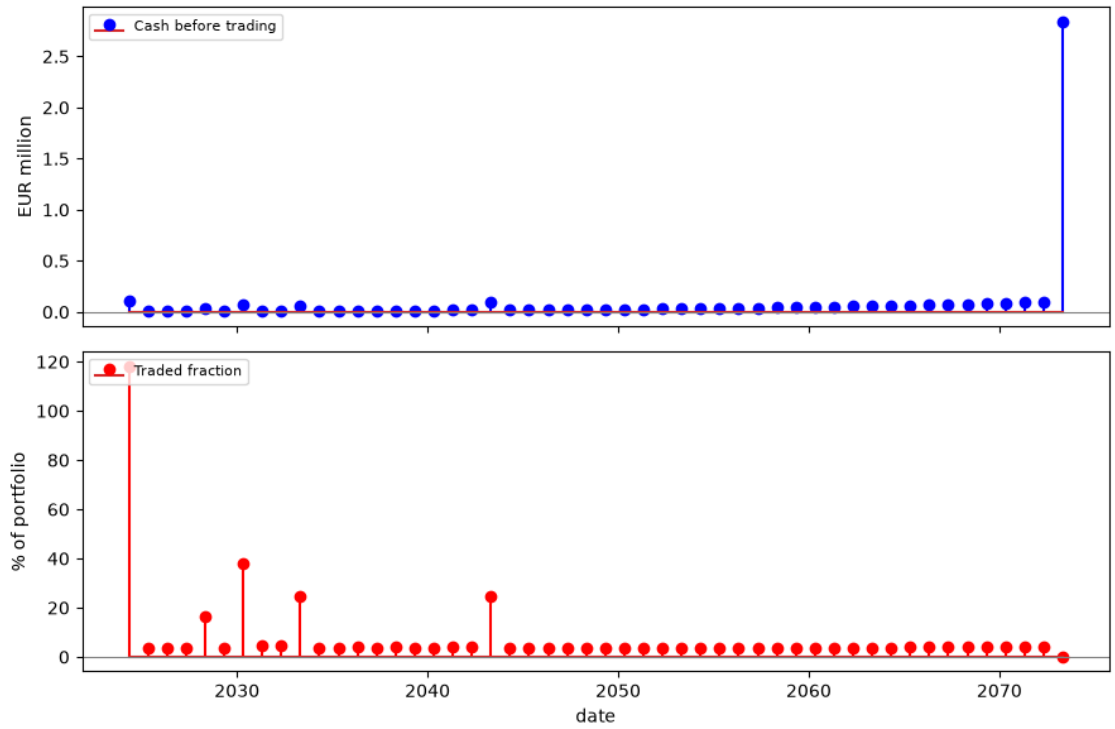
---

[22]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(9, 6), sharex=True)

ax1.stem(trading_log.index, trading_log["Cash before"] / 1e6, linefmt="b",␣
 ↪label="Cash before trading")
ax1.axhline(0, color="grey", linewidth=0.8)
ax1.set_ylabel("EUR million")
ax1.legend(loc="upper left", fontsize=8)

ax2.stem(trading_log.index, trading_log["Traded fraction"] * 100, linefmt="r",␣
 ↪label="Traded fraction")
ax2.axhline(0, color="grey", linewidth=0.8)
ax2.set_ylabel("% of portfolio")
ax2.set_xlabel("date")
ax2.legend(loc="upper left", fontsize=8)

fig.tight_layout()
plt.show()
```



The two panels are the same quantity seen twice: the traded fraction is just the cash balance divided by the portfolio it is spread across. Early periods trade a large share of a small portfolio; later ones trade a small share of a large one.

1.6 5. Checks

1.6.1 Cash is swept to zero

Trading exists to clear the bank account, so the balance after it should be zero in every period where there is a portfolio to trade against. The final period is the exception: by then every position has been liquidated, market value is zero, and the third branch of the rule leaves the proceeds sitting in cash.

---

[23]:
```
# trade() only acts when there is a portfolio to resize. In the final period␣
↪every position
# has been liquidated, so market value before trading is zero, the MV <= 0␣
↪branch is taken and
# the balance is left alone. That period is excluded here and reported on its␣
↪own.
tradable = trading_log["Market value before"] > 0
cash_left = trading_log.loc[tradable, "Cash after"].abs()
```

```
print(f"Periods with a portfolio to trade : {int(tradable.sum())} of␣
 ↪{len(trading_log)}")
print(f"Max |cash| left after trading     : {cash_left.max():.3e}")
print()

if (cash_left < 1e-6).all():
    print("PASSED - every tradable period ends with an empty bank account")
else:
    print("FAILED - cash left after trading in:")
    display(trading_log.loc[tradable & (trading_log["Cash after"].abs() >=␣
 ↪1e-6),
                            ["Cash before", "Market value before", "Cash␣
 ↪after"]])

assert (cash_left < 1e-6).all()
```

```
Periods with a portfolio to trade : 49 of 50
Max |cash| left after trading     : 4.511e-10

PASSED - every tradable period ends with an empty bank account
```

1.6.2 Trading does not create or destroy value

A trade moves value between the bank account and the portfolio, so the two together must be unchanged across the call:

$$C^{before} + MV^{before} = C^{after} + MV^{after}$$

---

[24]:
```
value_check = (trading_log["Cash before"] + trading_log["Market value before"]
               - trading_log["Cash after"] - trading_log["Market value after"])
scale = (trading_log["Cash before"].abs() + trading_log["Market value before"].
 ↪abs())

print(f"Max |residual|          : {value_check.abs().max():.3e}")
print(f"Max relative residual   : {(value_check / scale).abs().max():.3e}")
print()

if (value_check / scale).abs().max() < 1e-12:
    print("PASSED - trading is value neutral")
else:
    print("FAILED - value created or destroyed by trading:")
    display(pd.DataFrame({"Residual": value_check[(value_check / scale).abs()␣
 ↪>= 1e-12]}))
```

```
assert (value_check / scale).abs().max() < 1e-12
```

```
Max |residual|          : 5.821e-11
Max relative residual   : 1.529e-16

PASSED - trading is value neutral
```

The closing position, for comparison with a full `main.py` run:

---

[25]:
```
closing = pd.DataFrame({"Value": [
    float(bank_account[previous_date].iloc[0]),
    portfolio_market_value(eq_price_df, eq_units_df, bd_price_df, bd_units_df,␣
  ↪previous_date),
]}, index=["End cash", "End market value"])
closing.loc["Total"] = closing["Value"].sum()

display(closing)
```

	Value
End cash	2.844795e+06
End market value	0.000000e+00
Total	2.844795e+06
